

UNIVERSIDADE FEDERAL DE LAVRAS

Departamento de Física (DFI)

**SISTEMA EMBARCADO PARA DETECÇÃO DE
APOGEU E EJEÇÃO DE PARAQUEDAS EM FOGUETE**

Relatório Técnico de Projeto

Amanda Cristina Avelino

Lavras — MG

2026

Resumo

O presente trabalho apresenta o desenvolvimento de um sistema embarcado para detecção automática do apogeu e acionamento de um mecanismo de recuperação para foguetes experimentais. O objetivo do projeto foi implementar um protótipo capaz de identificar o momento em que o foguete atinge sua altitude máxima e, a partir dessa informação, acionar automaticamente um servomotor responsável pela abertura do compartimento do paraquedas, além de emitir um sinal sonoro por meio de um buzzer. O sistema foi desenvolvido utilizando um microcontrolador ESP32 programado no ambiente ESP-IDF, explorando os recursos do sistema operacional de tempo real FreeRTOS, como tarefas concorrentes, filas de comunicação (*Queues*) e notificações entre tarefas (*Task Notifications*). Para a aquisição dos dados foi utilizado o sensor barométrico BMP085, conectado ao ESP32 por meio do protocolo de comunicação I²C. A lógica de funcionamento foi implementada por meio de uma máquina de estados finitos, que monitora continuamente a variação da pressão atmosférica para detectar a subida, o apogeu e o início da descida do foguete. Com o objetivo de reduzir a influência do ruído presente nas medições, foi empregado um filtro de média móvel, proporcionando maior estabilidade às leituras e aumentando a confiabilidade do algoritmo de detecção. Os testes de bancada demonstraram o funcionamento adequado do sistema, evidenciando a correta comunicação entre os dispositivos, a detecção consistente do apogeu e o acionamento automático dos atuadores. Os resultados obtidos demonstram que a solução desenvolvida atende aos objetivos propostos e constitui uma base promissora para futuras aplicações em sistemas de recuperação de foguetes experimentais.

Palavras-chave: Sistemas Embarcados; ESP32; FreeRTOS; BMP085; Detecção de Apogeu; Servo Motor; Foguetes Experimentais.

1. Introdução e Motivação

O desenvolvimento de sistemas embarcados tem desempenhado um papel fundamental na evolução de diversas áreas da engenharia, possibilitando a automação de processos, o monitoramento em tempo real e o controle preciso de dispositivos eletrônicos. Esses sistemas estão presentes em aplicações industriais, automotivas, médicas, aeroespaciais e de telecomunicações, sendo caracterizados pela integração entre hardware e software dedicados à execução de funções específicas. A utilização de microcontroladores modernos, aliados a sistemas operacionais de tempo real (Real-Time Operating Systems – RTOS), permite o desenvolvimento de soluções robustas, eficientes e capazes de responder rapidamente a eventos externos.

No contexto aeroespacial, os sistemas embarcados desempenham um papel essencial em veículos lançadores e foguetes experimentais, sendo empregados em tarefas como aquisição de dados, telemetria, navegação, controle de voo e acionamento de mecanismos de recuperação. Em projetos acadêmicos e competições estudantis, esses sistemas tornam possível a implementação de soluções de baixo custo capazes de aumentar a segurança dos experimentos e reduzir a probabilidade de perda do veículo após o voo.

Um dos momentos mais críticos durante a trajetória de um foguete é o **apogeu**, definido como o ponto de maior altitude alcançada durante o voo. Após atingir esse ponto, o foguete inicia sua descida, tornando necessária a abertura do sistema de recuperação, normalmente composto por um paraquedas. Caso essa abertura ocorra de forma antecipada ou tardia, podem ocorrer falhas que comprometam a integridade estrutural do foguete, aumentando significativamente o risco de danos durante o impacto com o solo. Dessa forma, a detecção precisa do apogeu constitui um dos principais desafios no desenvolvimento de sistemas de recuperação automática.

Diversas técnicas podem ser empregadas para identificar o apogeu, utilizando sensores como acelerômetros, giroscópios, receptores GNSS e sensores barométricos. Entre essas alternativas, os sensores barométricos destacam-se pela simplicidade de implementação, baixo custo e boa precisão para aplicações experimentais, uma vez que permitem estimar a variação de altitude por meio da medição da pressão atmosférica. À medida que o foguete sobe, a pressão atmosférica diminui gradativamente; após o apogeu, durante a descida, essa pressão volta a aumentar. Essa característica permite identificar o momento da inversão do movimento utilizando algoritmos relativamente simples, desde que sejam adotadas técnicas adequadas para reduzir a influência do ruído presente nas medições.

Neste contexto, o presente trabalho propõe o desenvolvimento de um sistema embarcado para detecção automática do apogeu e acionamento de um mecanismo de recuperação para foguetes experimentais. O sistema foi implementado utilizando um microcontrolador ESP32, programado por meio do framework ESP-IDF e do sistema operacional de tempo real FreeRTOS. A aquisição dos dados foi realizada por um sensor barométrico BMP085, conectado ao microcontrolador através do protocolo de comunicação I²C. O algoritmo desenvolvido monitora continuamente a pressão atmosférica, aplicando uma máquina de estados finitos e um filtro de média móvel para aumentar a confiabilidade da detecção do apogeu. Após a confirmação do início da descida, o sistema aciona automaticamente um

servomotor responsável pela abertura do compartimento do paraquedas e um buzzer para sinalização sonora do evento.

Além de atender aos objetivos da disciplina de Sistemas Embarcados, o projeto proporciona a aplicação prática de diversos conceitos estudados ao longo do curso, incluindo comunicação serial, controle por modulação por largura de pulso (PWM), sistemas operacionais de tempo real, sincronização entre tarefas, aquisição de dados por sensores e desenvolvimento de algoritmos para tomada de decisão em tempo real. Dessa forma, o trabalho integra conhecimentos de eletrônica, programação e instrumentação, demonstrando a viabilidade da utilização de sistemas embarcados em aplicações voltadas ao desenvolvimento de tecnologias aeroespaciais de baixo custo.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema embarcado para detecção automática do apogeu de um foguete experimental, utilizando um sensor barométrico e um microcontrolador ESP32, capaz de acionar automaticamente um mecanismo de recuperação por meio de um servomotor e emitir uma sinalização sonora, aplicando os conceitos estudados na disciplina de Sistemas Embarcados.

2.2 Objetivos Específicos

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

- Desenvolver um sistema embarcado utilizando o microcontrolador ESP32 e o framework ESP-IDF;
- Implementar a comunicação entre o ESP32 e o sensor barométrico BMP085 por meio do protocolo I²C;
- Realizar a aquisição contínua dos dados de pressão atmosférica durante o funcionamento do sistema;
- Aplicar um filtro de média móvel para reduzir a influência de ruídos nas medições do sensor;
- Desenvolver um algoritmo capaz de detectar automaticamente o apogeu a partir da variação da pressão atmosférica;
- Implementar uma máquina de estados finitos para gerenciar as diferentes etapas de operação do sistema;
- Utilizar o sistema operacional de tempo real FreeRTOS para organizar a execução das tarefas concorrentes;
- Empregar filas (*Queues*) e notificações entre tarefas (*Task Notifications*) para sincronização e comunicação entre os diferentes módulos do software;
- Controlar um servomotor por meio de um sinal PWM para simular a abertura do compartimento do paraquedas;
- Acionar um buzzer como indicação sonora da detecção do apogeu;
- Validar o funcionamento do sistema por meio de testes de bancada, simulando as fases de subida e descida de um foguete experimental.

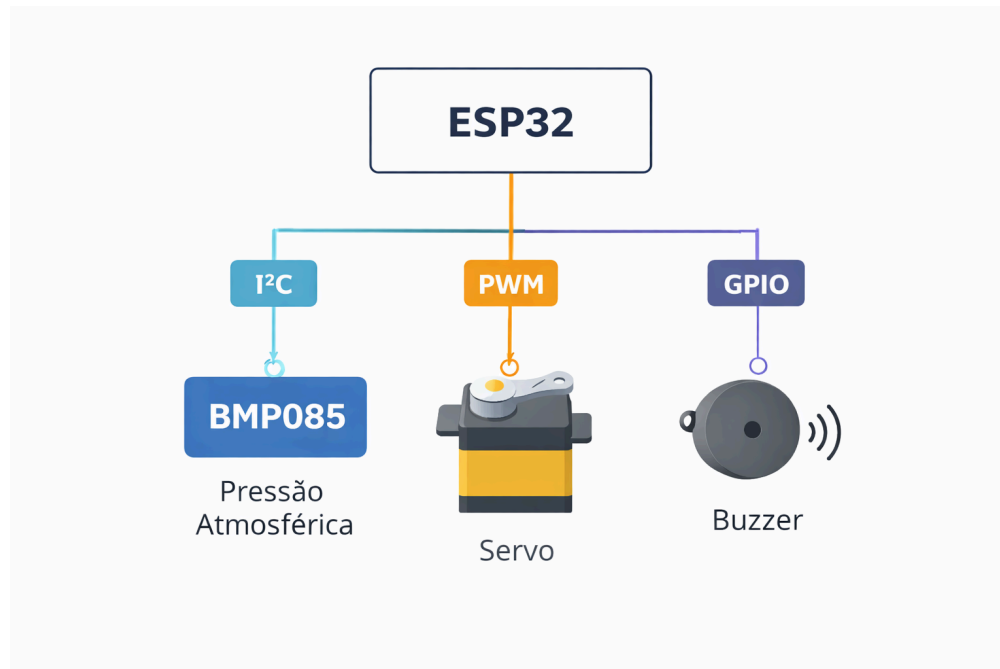


Figura 2.2 - Diagrama de blocos do sistema eletrônico mostrando o ESP32 conectado ao sensor BMP085 (pressão atmosférica), servo motor e buzzer. Fonte: Copilot

3. Fundamentação Teórica

O desenvolvimento de sistemas embarcados requer a integração de conhecimentos de eletrônica, programação e comunicação entre dispositivos. Neste projeto foram empregados conceitos relacionados a microcontroladores, sistemas operacionais de tempo real, sensores barométricos, protocolos de comunicação digital e técnicas de controle de atuadores. Nesta seção são apresentados os principais fundamentos teóricos que serviram de base para o desenvolvimento do sistema de detecção automática do apogeu.

3.1 Sistemas Embarcados

Um sistema embarcado é um sistema computacional desenvolvido para executar uma ou mais funções específicas dentro de um equipamento eletrônico. Diferentemente de computadores de propósito geral, os sistemas embarcados são projetados para operar de forma dedicada, apresentando baixo consumo de energia, elevada confiabilidade e capacidade de resposta em tempo real.

Esses sistemas são amplamente utilizados em aplicações industriais, automotivas, médicas, aeroespaciais e de automação residencial. Em muitas dessas aplicações, o sistema precisa interagir continuamente com sensores e atuadores, processando informações provenientes do ambiente e executando ações automaticamente.

Em projetos aeroespaciais, os sistemas embarcados desempenham funções como aquisição de dados, telemetria, controle de voo, navegação e acionamento de mecanismos de recuperação. A utilização desses sistemas em foguetes experimentais permite aumentar a segurança das missões e automatizar processos que, de outra forma, dependeriam exclusivamente de intervenção humana.

No presente trabalho, o sistema embarcado foi desenvolvido utilizando o microcontrolador ESP32, responsável pela leitura do sensor barométrico, processamento dos dados e acionamento dos dispositivos de recuperação.



Figura 3: Fluxograma simplificado do sistema eletrônico, destacando o fluxo de dados desde os sensores até os atuadores. Fonte: Copilot

3.1 Sistemas Embarcados

O ESP32 é um microcontrolador desenvolvido pela Espressif Systems, amplamente utilizado em aplicações de Internet das Coisas (IoT), automação e sistemas embarcados devido ao seu elevado desempenho e baixo custo. O dispositivo possui processador dual-core de até 240 MHz, memória RAM integrada, interfaces de comunicação digital e suporte nativo aos protocolos Wi-Fi e Bluetooth.

Além da elevada capacidade de processamento, o ESP32 dispõe de diversos periféricos internos, incluindo conversores analógico-digitais (ADC), temporizadores, módulos PWM, interfaces UART, SPI e I²C, tornando-o adequado para aplicações que exigem comunicação com sensores e controle de atuadores.

Neste projeto, o ESP32 foi utilizado como unidade central de processamento, sendo responsável por realizar a comunicação com o sensor BMP085 através do protocolo I²C, executar o algoritmo de detecção do apogeu utilizando o FreeRTOS e controlar o servomotor por meio de sinais PWM.

3.3 Sistema Operacional de Tempo Real (FreeRTOS)

O FreeRTOS é um sistema operacional de tempo real (Real-Time Operating System – RTOS) amplamente empregado em sistemas embarcados. Sua principal função é organizar a execução de múltiplas tarefas concorrentes, permitindo que diferentes atividades sejam executadas de forma eficiente e previsível.

Ao contrário da programação sequencial, em que todas as instruções são executadas em uma única rotina, o FreeRTOS divide o software em tarefas independentes (*Tasks*), que são gerenciadas por um escalonador (*Scheduler*). Cada tarefa possui uma prioridade associada, permitindo que atividades mais importantes sejam executadas preferencialmente quando necessário.

Além do gerenciamento de tarefas, o FreeRTOS disponibiliza mecanismos de comunicação e sincronização entre processos, como filas (*Queues*), semáforos (*Semaphores*), mutexes e notificações entre tarefas (*Task Notifications*).

Neste projeto foram utilizadas três tarefas principais:

- **TaskSensor**, responsável pela leitura periódica do sensor BMP085;
- **TaskProcess**, responsável pelo processamento dos dados e detecção do apogeu;
- **TaskActuators**, responsável pelo acionamento do servomotor e do buzzer.

A comunicação entre as tarefas foi implementada utilizando uma fila (*Queue*), enquanto o acionamento dos atuadores foi realizado através de notificações entre tarefas (*Task Notifications*), garantindo sincronização e reduzindo o tempo de resposta do sistema.

Task	Nome	Prioridade	Stack (bytes)	Responsabilidade
vTaskSensor	Task_Sensor	2	3072	Leitura e filtragem do BMP085
vTaskProcess	Task_Process	3	3072	Detecção de apogeu / máquina de estados
vTaskActuators	Task_Actuators	4	3072	Acionamento do servo e buzzer

3.4 Sensor Barométrico BMP085

O BMP085 é um sensor barométrico digital desenvolvido para medir pressão atmosférica e temperatura. A partir da pressão atmosférica medida, é possível estimar a altitude utilizando a equação barométrica, tornando esse sensor amplamente empregado em aplicações de navegação, drones e foguetes experimentais.

Durante a subida de um foguete, a pressão atmosférica diminui em função do aumento da altitude. Após o apogeu, quando o foguete inicia a descida, a pressão volta a aumentar. Dessa forma, o monitoramento contínuo da pressão permite identificar o instante em que ocorre a inversão do movimento vertical.

O sensor comunica-se com o microcontrolador por meio do protocolo I²C, fornecendo os dados digitalmente e dispensando o uso de conversores analógico-digitais externos.

Embora o módulo GY-801 utilizado neste projeto contenha outros sensores, apenas o BMP085 foi empregado para a aquisição das informações de pressão atmosférica.

Equação barométrica

A relação entre pressão atmosférica e altitude pode ser aproximada pela seguinte expressão:

$$h = 44330 \left(1 - \left(\frac{P}{P_0} \right)^{0.1903} \right)$$

em que:

- h é a altitude (m);
- P é a pressão medida (Pa);
- P_0 é a pressão ao nível de referência (Pa).

No presente trabalho, entretanto, a altitude não foi calculada explicitamente. O algoritmo baseou-se apenas na análise da variação da pressão atmosférica, reduzindo o custo computacional e simplificando a implementação.

3.5 Protocolo de Comunicação I²C

O protocolo Inter-Integrated Circuit (I²C) é um barramento de comunicação serial síncrona desenvolvido para permitir a comunicação entre circuitos integrados utilizando apenas duas linhas de dados.

O barramento é composto pelos sinais:

- SDA (Serial Data);
- SCL (Serial Clock).

Nesse protocolo, o microcontrolador atua como dispositivo mestre, enquanto os sensores funcionam como dispositivos escravos, cada um identificado por um endereço único.

No projeto desenvolvido, o ESP32 atua como mestre, realizando periodicamente a leitura dos registradores internos do BMP085 através do barramento I²C.

Tabela 3.5 – Componentes utilizados no projeto

Parâmetro	Valor no Projeto
Pino SDA	GPIO 21
Pino SCL	GPIO 22
Velocidade	100 kHz (modo Standard)
Endereço BMP085	0x77 (7 bits)
Pull-up interno	Habilitado via flag do driver

3.6 Modulação por Largura de Pulso (PWM) e Servomotor

A Modulação por Largura de Pulso (Pulse Width Modulation – PWM) é uma técnica utilizada para controlar dispositivos eletrônicos por meio da variação da largura dos pulsos de um sinal digital.

Os servomotores utilizam sinais PWM para posicionar seu eixo em um ângulo específico. Em geral, são empregados pulsos com frequência de aproximadamente 50 Hz, enquanto a largura do pulso determina a posição angular desejada.

Neste projeto, o periférico LEDC do ESP32 foi utilizado para gerar o sinal PWM responsável pelo acionamento do servomotor. Quando o apogeu é detectado, o algoritmo altera o ciclo de trabalho (*Duty Cycle*) do sinal PWM, provocando a rotação do servo e simulando a abertura do compartimento do paraquedas.

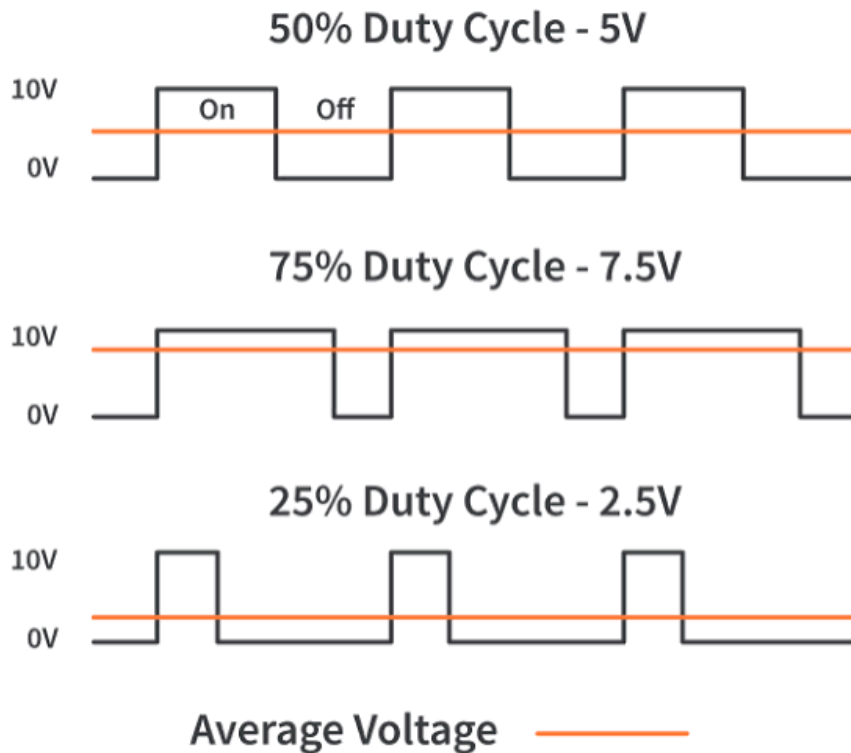


Figura 3.6 – Sinal PWM utilizado para o controle da posição de um servomotor. Fonte: CLUBE DO MAKER. *O que é PWM (Pulse Width Modulation)?* 2021. Disponível em: <https://clubedomaker.com/o-que-e-pwm-pulse-width-modulation>. Acesso em: 3 jul. 2026.

3.7 Máquina de Estados Finitos

Uma máquina de estados finitos (Finite State Machine – FSM) é um modelo computacional utilizado para representar sistemas cujo comportamento depende de um conjunto limitado de estados e das transições entre eles.

Nesse modelo, o sistema permanece em um determinado estado até que uma condição específica seja satisfeita, momento em que ocorre uma transição para outro estado.

A utilização de máquinas de estados facilita a implementação de sistemas embarcados, tornando o código mais organizado, modular e de fácil manutenção.

Neste projeto foram definidos quatro estados principais:

- **Armado**, no qual o sistema aguarda o início da subida;
- **Subindo**, durante o monitoramento contínuo da pressão atmosférica;
- **Apogeu Detectado**, quando ocorre o acionamento dos atuadores;
- **Finalizado**, impedindo novos disparos durante a descida.

Essa abordagem permitiu implementar um algoritmo robusto para identificação do apogeu, reduzindo a ocorrência de acionamentos indevidos causados por oscilações nas leituras do sensor.

4. Materiais e Métodos

Nesta seção são apresentados os materiais utilizados no desenvolvimento do projeto, bem como a metodologia adotada para a construção do protótipo, integração dos componentes eletrônicos, implementação do software embarcado e realização dos testes experimentais. A combinação entre hardware e software permitiu desenvolver um sistema capaz de detectar automaticamente o apogeu de um foguete experimental e acionar um mecanismo de recuperação de forma autônoma.

4.1 Materiais Utilizados

O protótipo foi desenvolvido utilizando componentes eletrônicos de baixo custo e amplamente empregados em aplicações de sistemas embarcados. A Tabela 4.1 apresenta os principais dispositivos utilizados durante o desenvolvimento.

Tabela 4.1 – Componentes utilizados no projeto

Componente	Quantidade	Função
ESP32 DevKit V1	1	Microcontrolador responsável pelo processamento do sistema
Módulo GY-801 (BMP085)	1	Medição da pressão atmosférica
Servo Motor SG90	1	Simulação da abertura do compartimento do paraquedas
Buzzer	1	Sinalização sonora da detecção do apogeu
Protoboard	1	Montagem do circuito
Jumpers	Diversos	Interligação dos componentes
Cabo USB	1	Alimentação e gravação do firmware

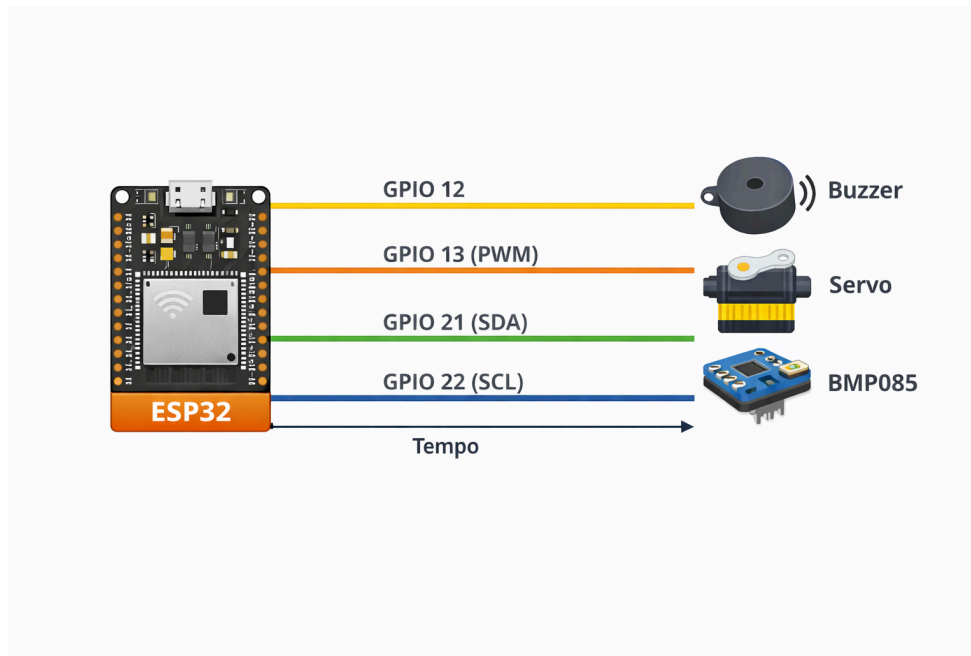
4.2 Arquitetura do Hardware

O sistema embarcado foi construído em torno do microcontrolador ESP32, responsável pela aquisição dos dados provenientes do sensor barométrico, processamento das informações e controle dos atuadores.

A comunicação entre o ESP32 e o sensor BMP085 foi realizada utilizando o protocolo I²C, permitindo a leitura periódica dos valores de pressão atmosférica. O servomotor foi conectado a um pino configurado para geração de sinais PWM, enquanto o buzzer foi ligado a uma saída digital utilizada para sinalização sonora após a detecção do apogeu.

A Figura 4.1 apresenta o diagrama simplificado da arquitetura do hardware desenvolvido.

Figura 4.1 – Arquitetura do hardware. Fonte: Copilot



4.3 Ligações Elétricas

A montagem do circuito foi realizada em uma protoboard, permitindo a integração entre os componentes de forma rápida e modular.

O sensor BMP085 foi conectado ao barramento I²C do ESP32, utilizando os pinos SDA e SCL. A alimentação foi fornecida diretamente pela placa ESP32, enquanto o servo motor foi conectado a uma saída PWM configurada através do periférico LEDC. O buzzer foi ligado a um pino digital para permitir sua ativação após a detecção do apogeu.

A Tabela 4.2 apresenta o mapeamento dos pinos utilizados.

Tabela 4.2 – Ligações do ESP32

ESP32	Dispositivo
GPIO 21	SDA do BMP085
GPIO 22	SCL do BMP085

GPIO 13	Servo Motor
GPIO 12	Buzzer
3,3 V	Alimentação do BMP085
GND	Terra comum

4.4 Desenvolvimento do Software

O software embarcado foi desenvolvido utilizando a linguagem C no ambiente ESP-IDF, que disponibiliza bibliotecas para controle dos periféricos do ESP32 e integração com o sistema operacional de tempo real FreeRTOS.

Durante o desenvolvimento, o código foi dividido em módulos responsáveis pela leitura do sensor, processamento dos dados e acionamento dos atuadores. Essa organização permitiu maior modularidade e facilitou os testes individuais de cada funcionalidade.

A comunicação com o sensor BMP085 foi implementada utilizando a API de comunicação I²C disponibilizada pelo ESP-IDF, enquanto o controle do servo motor foi realizado por meio do periférico LEDC, responsável pela geração dos sinais PWM.

Além disso, foram empregados mecanismos do FreeRTOS, como tarefas concorrentes, filas de comunicação e notificações entre tarefas, possibilitando uma arquitetura organizada e adequada para aplicações em tempo real.

4.5 Desenvolvimento Mecânico e Fabricação do Protótipo

Além do desenvolvimento eletrônico e do software embarcado, foi necessária a construção de uma estrutura mecânica capaz de acomodar todos os componentes do sistema de recuperação. Para isso, optou-se pela fabricação das peças utilizando a tecnologia de impressão 3D, devido ao seu baixo custo, rapidez na prototipagem e facilidade para realizar modificações durante o desenvolvimento.

O conjunto mecânico foi projetado para comportar o microcontrolador ESP32, o sensor barométrico BMP085, o servomotor responsável pelo acionamento do mecanismo de recuperação e os demais componentes eletrônicos utilizados no sistema.

A utilização da impressão 3D permitiu desenvolver um protótipo leve e personalizado, adequado às dimensões do foguete experimental e compatível com os requisitos do projeto. Além disso, a fabricação aditiva possibilitou a produção rápida de novas versões

das peças sempre que foram necessários ajustes de montagem ou de posicionamento dos componentes.

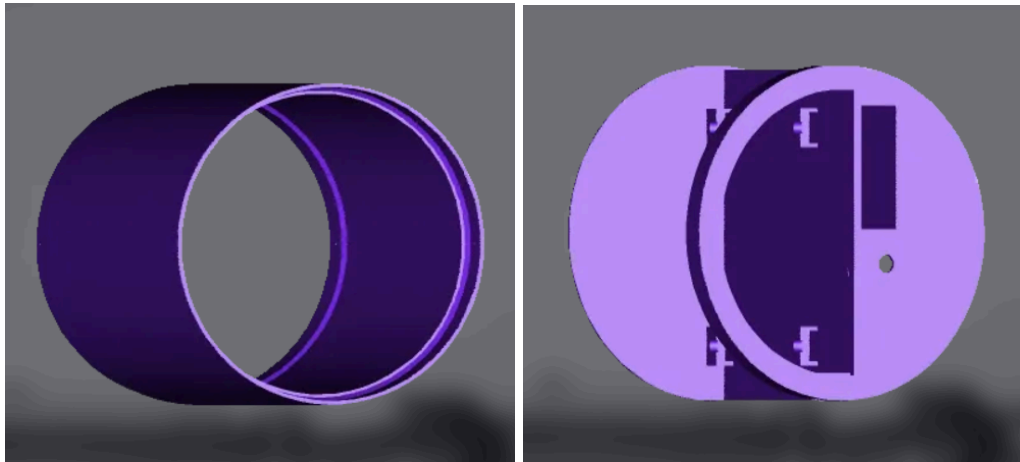
As peças foram modeladas em software de desenho tridimensional (CAD) e posteriormente convertidas para o formato STL, utilizado pelo software de fatiamento da impressora 3D. Após essa etapa, os modelos foram impressos utilizando filamento PLA, material amplamente empregado em prototipagem devido à sua facilidade de impressão, boa rigidez mecânica e baixo custo.

O conjunto produzido foi dividido em duas partes principais:

- compartimento eletrônico, responsável por alojar o ESP32, o sensor BMP085 e as conexões elétricas;
- compartimento do sistema de recuperação, contendo o servomotor e o mecanismo responsável pela abertura do compartimento do paraquedas.

Essa divisão facilita a montagem, a manutenção e futuras modificações no projeto.

Figuras 4.5 – Modelo CAD do sistema de recuperação





Fonte: Autoria própria.

Figura 4.5.2 – Protótipo montado



Fonte: Autoria própria.

Processo de fabricação

Após a modelagem das peças, foi realizada a impressão utilizando uma impressora 3D do tipo FDM (*Fused Deposition Modeling*). O processo consiste na deposição sucessiva de camadas de material termoplástico fundido até a obtenção da geometria desejada.

A escolha da tecnologia FDM ocorreu por sua ampla disponibilidade em laboratórios de prototipagem e pelo excelente custo-benefício para aplicações acadêmicas. Além disso, o processo permite fabricar componentes com geometrias complexas sem necessidade de usinagem convencional.

Após a impressão, as peças passaram por uma etapa de acabamento simples, consistindo na remoção de suportes e verificação dimensional, sendo posteriormente utilizadas na montagem do protótipo.

Vantagens da utilização da impressão 3D

A utilização da manufatura aditiva trouxe diversas vantagens para o desenvolvimento do projeto, entre as quais destacam-se:

- rápida fabricação de protótipos;
- baixo custo de produção;
- facilidade para realizar modificações no projeto;
- redução do tempo de desenvolvimento;
- fabricação de peças leves, adequadas para aplicações aeroespaciais experimentais;
- possibilidade de reproduzir o protótipo sempre que necessário.

4.5 Metodologia de Testes

A validação do sistema foi realizada por meio de testes de bancada, simulando o comportamento de um foguete experimental durante as fases de subida e descida.

Inicialmente, o sensor permaneceu em repouso para estabilização das leituras de pressão atmosférica. Em seguida, o conjunto contendo o ESP32 e o sensor foi elevado manualmente, simulando a ascensão do foguete. Durante esse movimento, o algoritmo monitorou continuamente a redução da pressão atmosférica.

Após atingir a maior altura possível no ambiente de teste, iniciou-se o movimento descendente. O aumento consecutivo da pressão foi identificado pelo algoritmo como o início da descida, caracterizando a detecção do apogeu. Nesse instante, o sistema acionou automaticamente o servomotor e o buzzer, simulando a abertura do compartimento do paraquedas e confirmando o funcionamento do sistema.

Os testes foram repetidos diversas vezes com o objetivo de ajustar os parâmetros do algoritmo, como o tamanho do filtro de média móvel, os limiares de detecção da subida e o número de leituras consecutivas necessárias para confirmar o início da descida. Esses ajustes permitiram reduzir a influência de oscilações nas leituras do sensor e aumentar a confiabilidade da detecção.

5. Desenvolvimento do Sistema

Nesta seção são apresentados os aspectos relacionados ao desenvolvimento do software embarcado implementado no ESP32. São descritas a arquitetura do sistema, a organização das tarefas utilizando o FreeRTOS, o algoritmo de detecção do apogeu, a comunicação entre os módulos e o acionamento dos atuadores. Durante o desenvolvimento foram realizados diversos testes e ajustes nos parâmetros do algoritmo, buscando aumentar a confiabilidade da detecção e reduzir a influência de oscilações presentes nas leituras do sensor.

5.1 Arquitetura do Software

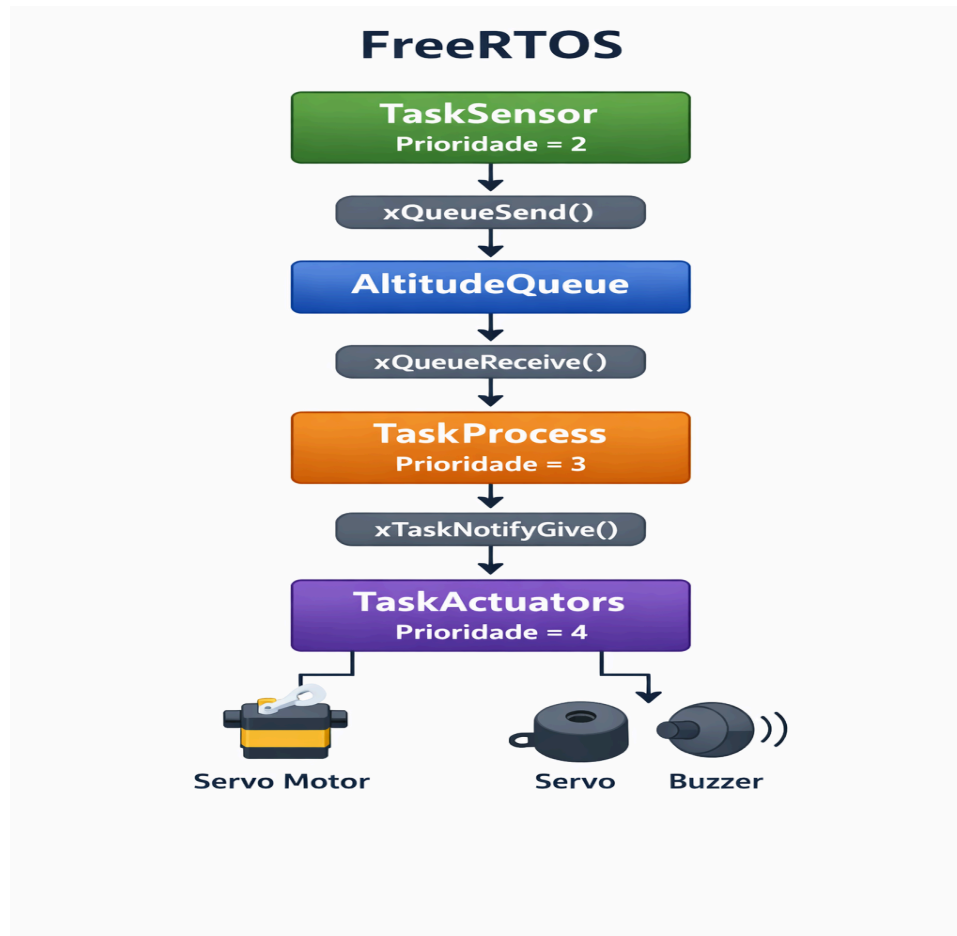
O software foi desenvolvido utilizando a linguagem C no ambiente ESP-IDF, explorando os recursos do sistema operacional de tempo real FreeRTOS. A adoção dessa arquitetura permitiu dividir o programa em tarefas independentes, facilitando a organização do código, aumentando sua modularidade e permitindo que diferentes atividades fossem executadas de forma concorrente.

O sistema foi organizado em três tarefas principais:

- **TaskSensor:** responsável pela aquisição dos dados provenientes do sensor BMP085.
- **TaskProcess:** responsável pelo processamento das leituras, aplicação do algoritmo de detecção do apogeu e gerenciamento da máquina de estados.
- **TaskActuators:** responsável pelo acionamento do servomotor e do buzzer após a confirmação do apogeu.

A comunicação entre as tarefas foi implementada por meio de uma fila (*Queue*), utilizada para transmitir os valores de pressão atmosférica adquiridos pelo sensor. Para o acionamento dos atuadores foi utilizada uma *Task Notification*, permitindo uma comunicação rápida entre a tarefa de processamento e a tarefa responsável pelo controle do servo motor e do buzzer.

Figura 5.1 – Arquitetura do software baseada no FreeRTOS



A criação das tarefas é realizada na função principal do programa (`app_main()`), conforme mostrado no trecho de código a seguir.

None

```
xTaskCreate(vTaskSensor,  
           "Task_Sensor",  
           3072,  
           NULL,  
           2,  
           NULL);
```

```
xTaskCreate(vTaskProcess,
```

```
        "Task_Process",  
        3072,  
        NULL,  
        3,  
        NULL);  
  
xTaskCreate(vTaskActuators,  
           "Task_Actuators",  
           3072,  
           NULL,  
           4,  
           &xActuatorTaskHandle);
```

Observa-se que a **TaskActuators** possui a maior prioridade, garantindo que o acionamento do sistema de recuperação ocorra imediatamente após a detecção do apogeu.

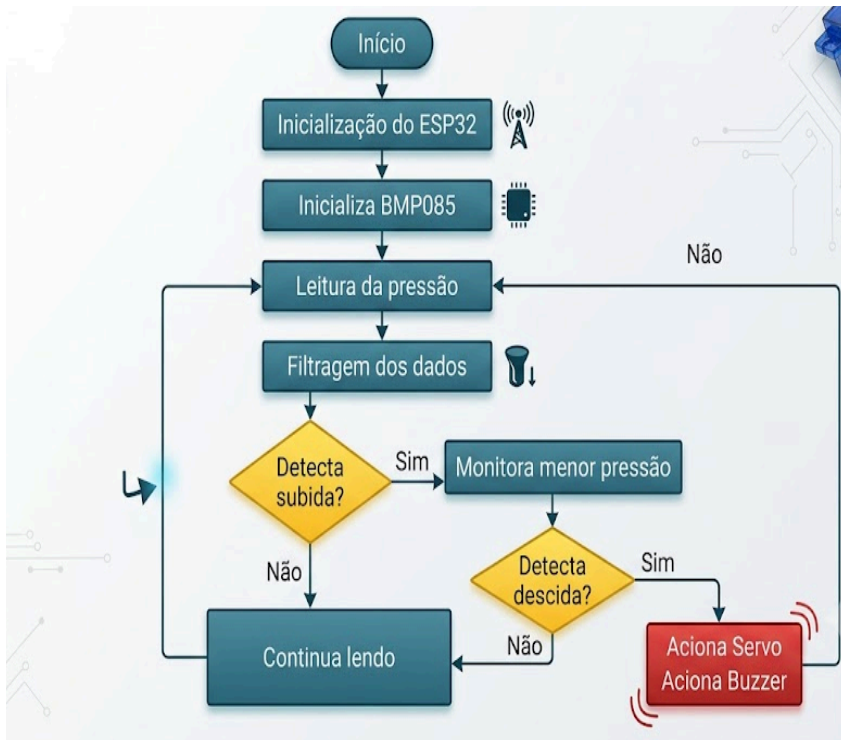
5.2 Fluxo Geral do Sistema

O funcionamento do sistema pode ser dividido em cinco etapas principais:

1. Inicialização do hardware.
2. Leitura contínua da pressão atmosférica.
3. Processamento das informações.
4. Detecção do apogeu.
5. Acionamento dos atuadores.

A Figura 5.2 apresenta o fluxograma geral do funcionamento do software.

Figura 5.2 – Fluxograma geral do sistema



5.3 Comunicação com o Sensor BMP085

A aquisição dos dados de pressão atmosférica foi realizada utilizando o protocolo I²C. Durante a inicialização do sistema, o barramento I²C é configurado e o sensor BMP085 é registrado como dispositivo escravo.

Após a configuração, a **TaskSensor** realiza leituras periódicas do sensor, enviando os valores obtidos para uma fila compartilhada com a tarefa de processamento.

O trecho abaixo apresenta parte da configuração do barramento I²C.

```

None
i2c_master_bus_config_t bus_config = {

    .i2c_port = I2C_NUM_0,

    .sda_io_num = I2C_MASTER_SDA,

    .scl_io_num = I2C_MASTER_SCL,

};
  
```

A leitura é realizada aproximadamente a cada 80 ms, garantindo uma taxa suficiente para acompanhar as variações de pressão durante o voo.

5.4 Filtragem das Leituras

Durante os testes verificou-se que as leituras do BMP085 apresentavam pequenas oscilações, mesmo com o sensor em repouso. Essas variações poderiam provocar falsas detecções do apogeu.

Para minimizar esse problema foi implementado um filtro de média móvel utilizando um vetor circular contendo as últimas leituras realizadas.

```
None
historico[indice] = ler_pressao_bruta();

indice = (indice + 1) % TAMANHO_FILTRO;
```

Em seguida, calcula-se a média das amostras armazenadas.

```
None
float soma = 0;

for(int i=0; i<TAMANHO_FILTRO; i++)
    soma += historico[i];

pressao_filtrada = soma/TAMANHO_FILTRO;
```

A utilização desse filtro proporcionou maior estabilidade às leituras e reduziu significativamente os acionamentos indevidos observados nas primeiras versões do algoritmo.

5.5 Algoritmo de Detecção do Apogeu

A lógica de detecção foi implementada utilizando uma máquina de estados finitos. O sistema inicia no estado **ARMADO**, aguardando a identificação da subida do foguete.

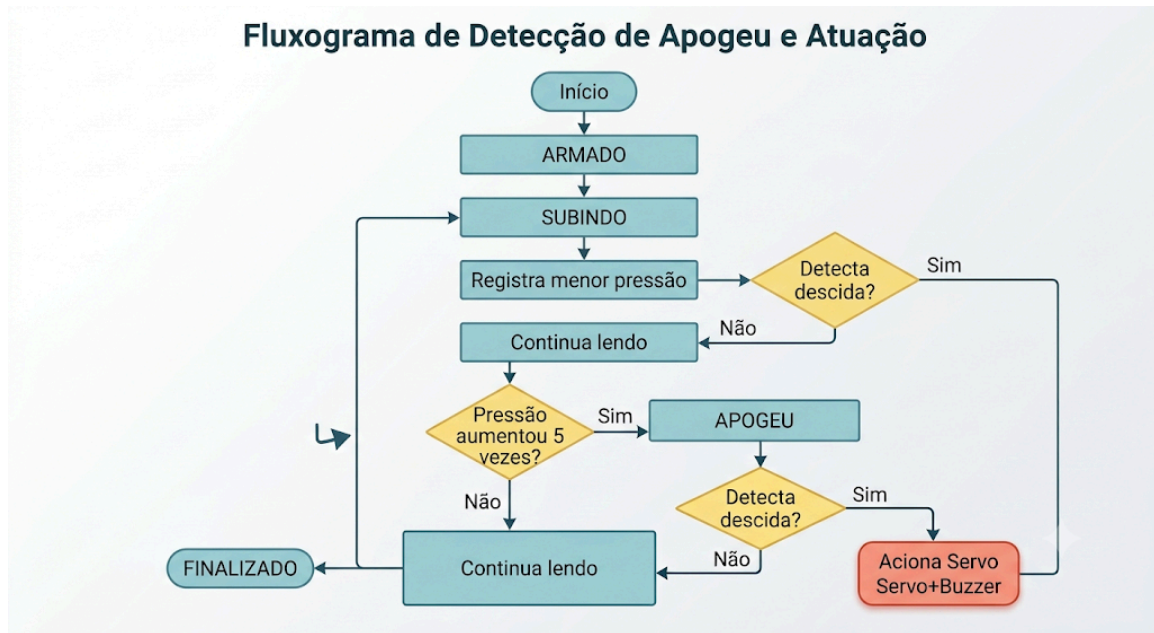
Quando a pressão atmosférica diminui continuamente em relação ao valor inicial, o sistema interpreta que o foguete iniciou sua ascensão e muda para o estado **SUBINDO**.

Durante essa etapa, o algoritmo registra continuamente o menor valor de pressão observado.

Quando a pressão passa a aumentar por diversas leituras consecutivas, o sistema interpreta que o foguete atingiu o apogeu e iniciou sua descida.

Nesse instante ocorre a transição para o estado **APOGEU**, responsável pelo acionamento do sistema de recuperação.

Figura 5.3 – Máquina de estados



O trecho responsável pela detecção da subida é apresentado a seguir.

```
None
if(pressaoAtual < (pressaoMesa - LIMIAR_SUBIDA))
{
    estadoVoo = 1;
}
```

Da mesma forma, o algoritmo confirma o início da descida apenas após várias leituras consecutivas, aumentando a confiabilidade da detecção.

```
None
if(pressaoAtual > menorPressao)
{
```

```
    confirmacaoDescida++;  
}
```

Essa estratégia evita disparos causados por pequenas oscilações do sensor.

5.6 Acionamento dos Atuadores

Após a confirmação do apogeu, a **TaskProcess** envia uma notificação para a **TaskActuators**, responsável pelo acionamento do sistema de recuperação.

O servomotor é controlado utilizando o periférico LEDC do ESP32, configurado para gerar sinais PWM com frequência de 50 Hz.

```
None  
ledc_set_duty(LEDC_MODE,  
              LEDC_CHANNEL,  
              SERVO_MAX_DUTY);  
  
ledc_update_duty(LEDC_MODE,  
                 LEDC_CHANNEL);
```

Após a movimentação do servo motor, o buzzer é ativado para emitir um sinal sonoro indicando que o sistema detectou corretamente o apogeu.

5.7 Ajustes e Calibração do Algoritmo

Durante o desenvolvimento foram realizados diversos testes de bancada para avaliar o comportamento do sistema. Inicialmente, observou-se que pequenas oscilações nas leituras do sensor provocavam falsas detecções da subida e do apogeu.

Para solucionar esse problema, foram realizados ajustes no algoritmo, incluindo:

- implementação de um filtro de média móvel;
- definição de um limiar mínimo para caracterizar a subida;
- exigência de múltiplas leituras consecutivas para confirmar o início da descida;
- ajuste do intervalo de amostragem do sensor;
- calibração do número de amostras utilizadas na média móvel.

Esses ajustes permitiram aumentar significativamente a confiabilidade do sistema durante os testes, garantindo que o servomotor fosse acionado apenas após a identificação consistente do apogeu.

6. Resultados e Discussão

Após a implementação do sistema embarcado, foram realizados diversos testes de bancada com o objetivo de validar o funcionamento do hardware e do software. Os ensaios consistiram na simulação manual da trajetória de um foguete experimental, elevando e abaixando o conjunto formado pelo ESP32 e pelo sensor barométrico, de modo a reproduzir as fases de subida, apogeu e descida.

Durante os testes, foi possível avaliar individualmente cada módulo do sistema, desde a aquisição dos dados do sensor até o acionamento do servomotor responsável pela abertura do compartimento do paraquedas. A realização desses ensaios também permitiu identificar limitações do algoritmo inicialmente desenvolvido, possibilitando sua otimização por meio de sucessivas etapas de calibração.

6.1 Validação da Comunicação com o Sensor

O primeiro teste realizado consistiu em verificar a comunicação entre o ESP32 e o sensor BMP085 utilizando o protocolo I²C.

Inicialmente, foram observadas dificuldades na leitura correta dos registradores do sensor, ocasionadas principalmente pela configuração do barramento I²C e pela adaptação da biblioteca ao ambiente ESP-IDF. Após a correção da configuração dos dispositivos e da rotina de comunicação, o sistema passou a realizar leituras contínuas da pressão atmosférica.

A Figura 6.1 apresenta um exemplo das leituras registradas no monitor serial.

Figura 6.1 – Leituras da pressão atmosférica obtidas pelo BMP085.

```

W (11844) i2c.common: GPIO 22 is not usable, maybe conflict with others
E (11844) i2c.master: I2C hardware timeout detected
W (11864) i2c.common: GPIO 21 is not usable, maybe conflict with others
W (11864) i2c.common: GPIO 22 is not usable, maybe conflict with others
I (11904) FOGUETE_amand: Pressao: 33960.5 | Estado: 0 | Quedas: 0/2
W (11904) FOGUETE_amand: =====
W (11904) FOGUETE_amand: === SUBIDA DETECTADA (Foguete voando) ===
W (11904) FOGUETE_amand: =====
I (12004) FOGUETE_amand: Pressao: 33961.1 | Estado: 1 | Quedas: 0/2
I (12094) FOGUETE_amand: Pressao: 33961.6 | Estado: 1 | Quedas: 1/2
E (12094) FOGUETE_amand: !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
E (12094) FOGUETE_amand: !!! APOGEU DETECTADO! EXPULSANDO PARAQUEDAS !!!
E (12094) FOGUETE_amand: !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```

6.2 Oscilações nas Leituras do Sensor

Após a implementação da leitura contínua do sensor, verificou-se que os valores de pressão apresentavam pequenas oscilações mesmo quando o sistema permanecia completamente parado.

Embora essas variações fossem relativamente pequenas, elas eram suficientes para provocar falsas interpretações do algoritmo, que identificava incorretamente movimentos de subida ou descida.

Esse comportamento está relacionado às limitações naturais do sensor barométrico, bem como às pequenas variações ambientais e ao ruído presente nas medições.

Para reduzir esse efeito, foi implementado um filtro de média móvel utilizando as últimas leituras realizadas pelo sensor.

Após a aplicação desse filtro, observou-se uma redução significativa das oscilações, proporcionando maior estabilidade aos valores utilizados pelo algoritmo de detecção.

6.3 Problemas na Detecção da Subida

Durante os primeiros testes do algoritmo, observou-se que a detecção da subida apresentava comportamento inconsistente.

Em algumas execuções o sistema detectava corretamente o início da ascensão, enquanto em outras permanecia no estado inicial mesmo após a movimentação do sensor.

Esse comportamento foi atribuído à escolha inadequada do limiar utilizado para caracterizar a diminuição da pressão atmosférica.

Foram realizados diversos testes modificando esse parâmetro até encontrar um valor capaz de distinguir pequenas oscilações do sensor de uma subida real.

Além disso, verificou-se que movimentos muito lentos durante os testes dificultavam a identificação da subida, tornando necessária uma movimentação mais contínua para reproduzir adequadamente o comportamento esperado de um foguete.

6.4 Problemas na Detecção do Apogeu

Após resolver a identificação da subida, um novo problema foi observado durante os testes.

O algoritmo detectava corretamente a subida, porém acionava o servo motor imediatamente após atingir a maior altura, antes mesmo que o sensor fosse movimentado para baixo.

A análise dos registros do monitor serial mostrou que pequenas oscilações na pressão atmosférica eram interpretadas como início da descida.

Como consequência, o sistema acionava o mecanismo de recuperação de forma prematura.

Para solucionar esse problema, foram implementadas duas modificações principais:

- atualização contínua do menor valor de pressão registrado durante toda a subida;
- confirmação da descida apenas após diversas leituras consecutivas indicando aumento da pressão.

Essa estratégia tornou o algoritmo significativamente mais robusto, eliminando praticamente todos os disparos indevidos observados durante os testes.

6.5 Ajuste dos Parâmetros do Algoritmo

Diversos parâmetros do sistema foram calibrados experimentalmente ao longo do desenvolvimento.

Entre os principais ajustes realizados destacam-se:

- número de amostras utilizadas na média móvel;
- intervalo entre leituras do sensor;
- limiar mínimo para identificação da subida;
- número de leituras consecutivas utilizadas para confirmar o início da descida.

Esses parâmetros foram modificados diversas vezes até que o sistema apresentasse comportamento consistente durante sucessivos testes de bancada.

O processo de calibração foi fundamental para aumentar a confiabilidade da detecção do apogeu.

6.6 Acionamento do Servo Motor

Após a estabilização do algoritmo, foram realizados testes para validar o acionamento do servomotor.

Inicialmente, verificou-se que o servo permanecia corretamente na posição inicial e era acionado apenas após a confirmação do apogeu.

O controle do servo foi realizado por meio do periférico LEDC do ESP32, utilizando sinais PWM com frequência de 50 Hz.

Os testes demonstraram que o servomotor respondeu adequadamente ao comando enviado pela tarefa responsável pelos atuadores.

6.7 Problemas Encontrados com o Buzzer

Durante a fase de validação dos atuadores foram observadas dificuldades relacionadas ao buzzer.

Inicialmente foi utilizado um buzzer do tipo passivo, que produzia apenas pequenos pulsos sonoros de baixa intensidade quando acionado diretamente por um pino digital do ESP32.

Após a substituição do componente por outro modelo, verificou-se que o novo buzzer apresentava comportamento diferente, exigindo adaptações na forma de acionamento.

Apesar dessas dificuldades, o sistema foi capaz de fornecer uma indicação sonora durante os testes, permitindo confirmar a execução do algoritmo de detecção do apogeu.

Esses testes evidenciaram a importância de identificar corretamente o tipo de buzzer empregado (ativo ou passivo), uma vez que cada modelo possui características distintas de funcionamento.

6.8 Funcionamento Integrado do Sistema

Após a conclusão dos ajustes, foram realizados testes integrando todos os módulos desenvolvidos.

Durante os ensaios, o sistema executou corretamente a sequência prevista:

1. inicialização do hardware;
2. leitura contínua da pressão atmosférica;
3. identificação da subida;

4. monitoramento da menor pressão registrada;
5. confirmação do início da descida;
6. acionamento do servomotor;
7. emissão do sinal sonoro.

Os resultados obtidos demonstraram que a arquitetura baseada em FreeRTOS permitiu a execução concorrente das diferentes tarefas do sistema, garantindo comunicação eficiente entre os módulos e resposta adequada ao evento de detecção do apogeu.

Embora os testes tenham sido realizados em bancada, simulando manualmente o perfil de voo de um foguete, os resultados indicam que a estratégia desenvolvida apresenta potencial para aplicação em sistemas reais de recuperação, desde que sejam realizados testes adicionais em condições reais de lançamento.

6.9 Considerações sobre as Limitações

Apesar dos resultados satisfatórios, algumas limitações foram identificadas durante o desenvolvimento do projeto.

Os testes foram realizados em ambiente controlado, utilizando movimentação manual do sensor, o que não reproduz integralmente as acelerações e vibrações presentes em um lançamento real.

Além disso, a estratégia adotada baseia-se exclusivamente na pressão atmosférica. Em aplicações práticas, a fusão de dados provenientes de acelerômetros, giroscópios e sensores GNSS pode aumentar ainda mais a confiabilidade da detecção do apogeu.

Outra limitação observada foi a sensibilidade do algoritmo aos parâmetros de calibração, evidenciando a necessidade de ajustes específicos para diferentes condições ambientais e perfis de voo.

7. Conclusão

O presente trabalho teve como objetivo desenvolver um sistema embarcado capaz de detectar automaticamente o apogeu de um foguete experimental e acionar um mecanismo de recuperação utilizando um servomotor, empregando como plataforma de desenvolvimento o microcontrolador ESP32 e o sistema operacional de tempo real FreeRTOS.

Os resultados obtidos demonstraram que o sistema desenvolvido foi capaz de realizar a leitura contínua da pressão atmosférica por meio do sensor BMP085, processar as informações em tempo real e identificar corretamente o momento em que ocorreu a inversão do movimento, correspondente ao apogeu do foguete. Após essa identificação, o sistema acionou automaticamente o servomotor responsável pela simulação da abertura

do compartimento do paraquedas e emitiu uma sinalização sonora através do buzzer, atendendo ao funcionamento esperado.

Ao longo do desenvolvimento foram encontrados diversos desafios técnicos, principalmente relacionados à estabilidade das leituras do sensor barométrico e à calibração do algoritmo de detecção. Pequenas oscilações na pressão atmosférica provocavam falsas interpretações da subida e da descida, exigindo a implementação de um filtro de média móvel e de critérios adicionais de validação para confirmar o apogeu. O processo de testes e ajustes mostrou-se fundamental para aumentar a confiabilidade do sistema, evidenciando a importância da etapa de validação em projetos de sistemas embarcados.

Do ponto de vista da disciplina, o projeto permitiu aplicar diversos conceitos estudados ao longo do semestre, como comunicação I²C, geração de sinais PWM, programação em linguagem C utilizando o ESP-IDF, desenvolvimento de aplicações com FreeRTOS, criação de tarefas concorrentes, utilização de filas para comunicação entre tarefas e sincronização por meio de notificações. A implementação de uma máquina de estados finitos também contribuiu para tornar o software mais organizado, modular e de fácil manutenção.

Embora os testes tenham sido realizados em bancada, simulando manualmente o perfil de voo do foguete, os resultados obtidos demonstraram o potencial da solução proposta para aplicações em sistemas de recuperação de foguetes experimentais. A arquitetura desenvolvida pode ser expandida para aplicações reais mediante a realização de testes em voo e a integração com sensores adicionais.

Como trabalhos futuros, recomenda-se a implementação de algoritmos de fusão de sensores utilizando acelerômetros e giroscópios presentes no módulo GY-801, aumentando a robustez da detecção do apogeu. Também podem ser incorporados recursos como armazenamento dos dados em cartão microSD, transmissão de telemetria por rádio, utilização de filtros mais avançados, como o filtro de Kalman, e a validação do sistema em lançamentos reais de foguetes. Além disso, a fabricação de um compartimento para paraquedas utilizando impressão 3D permitirá avaliar o desempenho do mecanismo de recuperação em condições mais próximas das encontradas em aplicações práticas.

De forma geral, o projeto atingiu os objetivos propostos, demonstrando que é possível desenvolver um sistema embarcado de baixo custo, confiável e modular para detecção automática do apogeu e acionamento de mecanismos de recuperação, integrando conceitos de eletrônica, programação e sistemas de tempo real em uma aplicação de interesse para a área aeroespacial.

8. Referências Bibliográficas

1. ESPRESSIF SYSTEMS. **ESP-IDF Programming Guide**. Disponível em: <https://docs.espressif.com/projects/esp-idf/>. Acesso em: xx xxx. 2026.

2. BARRY, Richard. **Mastering the FreeRTOS Real Time Kernel**. Real Time Engineers Ltd.
3. BOSCH SENSORTEC. **BMP085 Digital Pressure Sensor Datasheet**.
4. TANENBAUM, Andrew S. **Sistemas Operacionais Modernos**. Pearson.
5. STALLINGS, William. **Arquitetura e Organização de Computadores**. Pearson.
6. OGATA, Katsuhiko. **Engenharia de Controle Moderno**. Pearson.
7. VALVANO, Jonathan W. **Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers**. Cengage Learning.
8. STEVENSTURGIS. *New Parachute System 2 Servo Drogue Chute Nose Cone Water Rockets*. Cults3D, 2025. Disponível em: [Cults3D](#). Acesso em: 3 jul. 2026.